

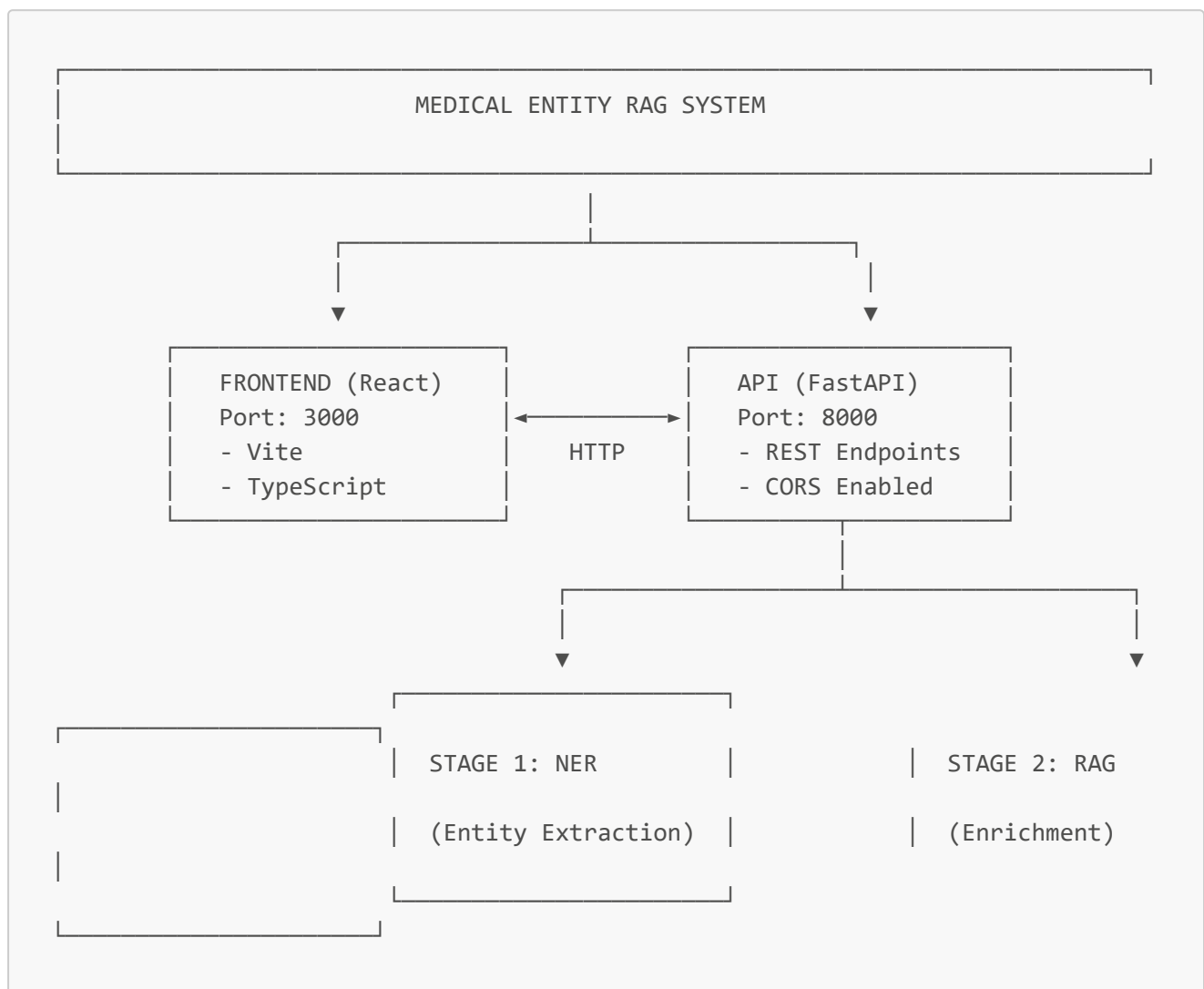
Medical Entity RAG System Architecture

Overview

This system implements a two-stage pipeline for medical entity extraction and clinical note analysis:

1. **NER (Named Entity Recognition)** - Extraction of medical entities from clinical notes
2. **RAG (Retrieval-Augmented Generation)** - Enrichment and contextualization using SNOMED-CT ontology

Architecture Diagram



Stage 1: NER (Named Entity Recognition)

Purpose

Extract medical entities from unstructured clinical notes with high recall (>99%) to ensure no critical medical information is missed.

Components

1.1 Multi-Worker Assembly

Located in: [ofarres/backend/src/NER/](#)

Workers:

- **OntologyNER** ([ontology_ner.py](#))
 - Exact dictionary-based matching using SNOMED-CT ontology
 - FlashText for O(n) performance
 - Automatic variation generation (plurals, head words)
 - **Confidence:** ★ ★ ★ ★ ★ High
- **ScispaCyNER** ([spacy_ner.py](#))
 - Transformer-based model (SciBERT)
 - Detects entities not in dictionary
 - Model: [en_core_sci_scibert](#)
 - **Confidence:** ★ ★ ★ Medium
- **AcronymNER** ([acronym_ner.py](#))
 - Specialized in medical acronyms (CT, MRI, NIHSS)
 - Case-sensitive with boundary detection
 - Stopword-aware filtering
 - **Confidence:** ★ ★ ★ ★ ★ High

1.2 5-Step Post-Processing Pipeline

Located in: [ofarres/backend/src/NER/postprocessor/](#)

Input: Clinical Note Text



```
STEP 01: Harvester (01_gather_assembly.py)
- Execute all NER workers in parallel
- Merge duplicate detections from multiple workers
- Output: Raw assembly with source attribution
```



```
STEP 02: Classifier (02_assign_ranks.py)
- Assign confidence tiers based on worker consensus
  • TIER 1 (Elite): Acronyms OR (Ontology + SciBERT)
  • TIER 2 (Gold): Ontology only
  • TIER 3 (Bronze): SciBERT only
```

- Output: Ranked entities with priority field



STEP 03: Safe Deduplication (03_safe_deduplication.py)

- Resolve overlapping entities ("Russian Doll" problem)
- Dictionary Sovereign + Coexistence strategy
 - High-confidence containers absorb nested low-confidence
 - Rank protection: nested high-confidence coexist
- Output: Deduplicated entities



STEP 04: Linguistic Filter (04_linguistic_filter.py)

- Remove syntactic noise from Tier 3 (low confidence)
- Rules: Headers, Ghosts, Lonely Modifiers
- Auto-pass: Tier 1 & 2 (dictionary-backed)
- Uses: spaCy (en_core_web_sm) for POS tagging
- Output: Linguistically clean entities



STEP 05: Semantic Judge (05_semantic_judge.py)

- Filter semantically irrelevant "Hard Noise"
- Hybrid approach:
 - Blacklist: Common clinical non-entities
 - Cross-encoder: Contrastive medical relevance scoring
- Model: cross-encoder/ms-marco-MiniLM-L-6-v2
- Auto-pass: Tier 1 & 2
- Output: Final semantically validated entities



Final Entity List
(JSON with annotations)

1.3 Orchestrator

File: ofarres/backend/src/NER/A_pipeline_orchestrator.py

Responsibilities:

- Execute all 5 pipeline steps sequentially
- Audit performance after each step
- Calculate RAG-friendly metrics (Recall, Precision, F1)
- Generate performance dashboard

Output Format:

```
[
  {
    "note_id": "1",
    "annotations": [
      {
        "start": 75,
        "end": 87,
        "text": "hypertension",
        "source": ["OntologyExact", "SBert"],
        "priority": 1,
        "semantic_score": 0.8234
      }
    ]
  }
]
```

Performance Metrics

RAG-Friendly Evaluation:

- **True Positive Criteria:**
 1. Physical overlap (IoU > 0.1)
 2. Text containment (GT $\subseteq$ Pred OR Pred $\subseteq$ GT)
 3. 1-to-1 matching (prevents bad merges)

Current Results:

Step Name	Entities	Recall	Precision	F1
01_gather_assembly	394	100.00%	24.62%	0.3951
02_assign_ranks	394	100.00%	24.62%	0.3951
03_safe_deduplication	389	100.00%	24.94%	0.3992
04_linguistic_filter	361	100.00%	26.87%	0.4236
05_semantic_judge	360	100.00%	26.94%	0.4245

Key Achievement: 100% Recall maintained throughout the entire pipeline

Stage 2: RAG (Retrieval-Augmented Generation)

Purpose

Enrich extracted entities with contextual medical knowledge from SNOMED-CT ontology to support clinical decision-making and information retrieval.

Architecture

RAG Pipeline (3 Phases)

Input: Extracted Entities from NER Stage



PHASE 1: RETRIEVAL (01_Retrieval/)

1.1 Entity Linking

- Map extracted text to SNOMED-CT concept IDs
- Semantic similarity using SapBERT embeddings
- FAISS index for fast nearest neighbor search

1.2 Ontology Retrieval

- Load multilingual SNOMED-CT ontology
- Retrieve concept definitions, relationships
- Extract: FSN, synonyms, semantic tags

Index: assets/ontology/ontology_rag.index (FAISS)

Data: ontology/multilingual_ontology.json



PHASE 2: AUGMENTATION (02_Augmentation/)

2.1 Context Enrichment

- Add hierarchical relationships (IS-A, PART-OF)
- Include related concepts and co-occurrences
- Temporal and anatomical context

2.2 Clinical Narratives

- Link to common clinical presentations
- Diagnostic criteria and guidelines
- Treatment protocols (if applicable)

2.3 Multi-lingual Support

- English, Spanish, Catalan terms
- Cross-language concept mapping



PHASE 3: GENERATION (03_Generation/)

3.1 Context Assembly

- Combine entity + ontology knowledge

- Structure for downstream LLM consumption

3.2 Response Formatting

- Structured JSON for API responses
- Markdown for human-readable reports
- Citations and provenance tracking

Integration: Can be extended with LLM (GPT-4, Claude) for natural language generation and clinical reasoning



Enriched Clinical Knowledge Graph

Components

2.1 Retrieval Module

Location: [ofarres/backend/src/RAG/01_Retrieval/](#)

Purpose: Map entities to standardized medical concepts

Technologies:

- **FAISS:** Fast similarity search
- **SapBERT:** Biomedical entity embeddings
- **SNOMED-CT:** Medical ontology (400k+ concepts)

Input: Entity text (e.g., "acute stroke")

Output: Concept ID + metadata

2.2 Augmentation Module

Location: [ofarres/backend/src/RAG/02_Augmentation/](#)

Purpose: Enrich concepts with contextual relationships

Features:

- Hierarchical concept navigation
- Related concept discovery
- Clinical narrative generation
- Multi-lingual term expansion

2.3 Generation Module

Location: [ofarres/backend/src/RAG/03_Generation/](#)

Purpose: Format enriched data for consumption

Outputs:

- Structured JSON (for APIs)
 - Markdown reports (for humans)
 - Knowledge graph visualizations
-

API Layer

FastAPI Service

Location: ofarres/api/

Port: 8000

Endpoints

Health Check

```
GET /api/v1/health
```

Returns API status and version information.

Get All Notes

```
GET /api/v1/notes
```

Returns list of all clinical notes.

Get Note with Entities

```
GET /api/v1/notes/{note_id}
```

Returns specific note with extracted entities.

Analyze Note

```
POST /api/v1/notes/analyze
```

Body:

```
{  
  "text": "Patient presents with acute ischemic stroke..."  
}
```

Response:

```
{
  "entities": [
    {
      "text": "ischemic stroke",
      "start": 25,
      "end": 40,
      "concept_id": "422504002",
      "confidence": 0.95,
      "tier": 1
    }
  ],
  "processing_time_ms": 45,
  "model_version": "v1.0.0"
}
```

Get Entities for Note

```
GET /api/v1/entities/{note_id}
```

Returns entities extracted from a specific note.

Run Benchmark

```
POST /api/v1/benchmark/run
```

Executes NER pipeline benchmark on test dataset.

Services Architecture

```
api/
├── main.py           # FastAPI app initialization
├── routers/          # Route handlers
│   ├── health.py     # Health endpoints
│   ├── notes.py       # Note CRUD operations
│   ├── entities.py    # Entity extraction
│   └── benchmark.py   # Benchmark execution
├── services/         # Business logic layer
│   ├── note_service.py # Note data management
│   ├── entity_service.py # Entity extraction logic
│   └── benchmark_service.py # Benchmark orchestration
```


Frontend Application

Technology Stack

Location: ofarres/frontend/

Port: 3000

Stack:

- **Framework:** React 19
- **Build Tool:** Vite
- **Language:** TypeScript
- **Routing:** React Router DOM
- **HTTP Client:** Axios
- **Icons:** Lucide React
- **Markdown:** React Markdown

Features

1. Note Browser

- List all clinical notes
- Search and filter functionality
- Note detail view with highlighting

2. Entity Explorer

- Visualize extracted entities
- Color-coded by confidence tier
- Interactive entity details

3. Analysis Dashboard

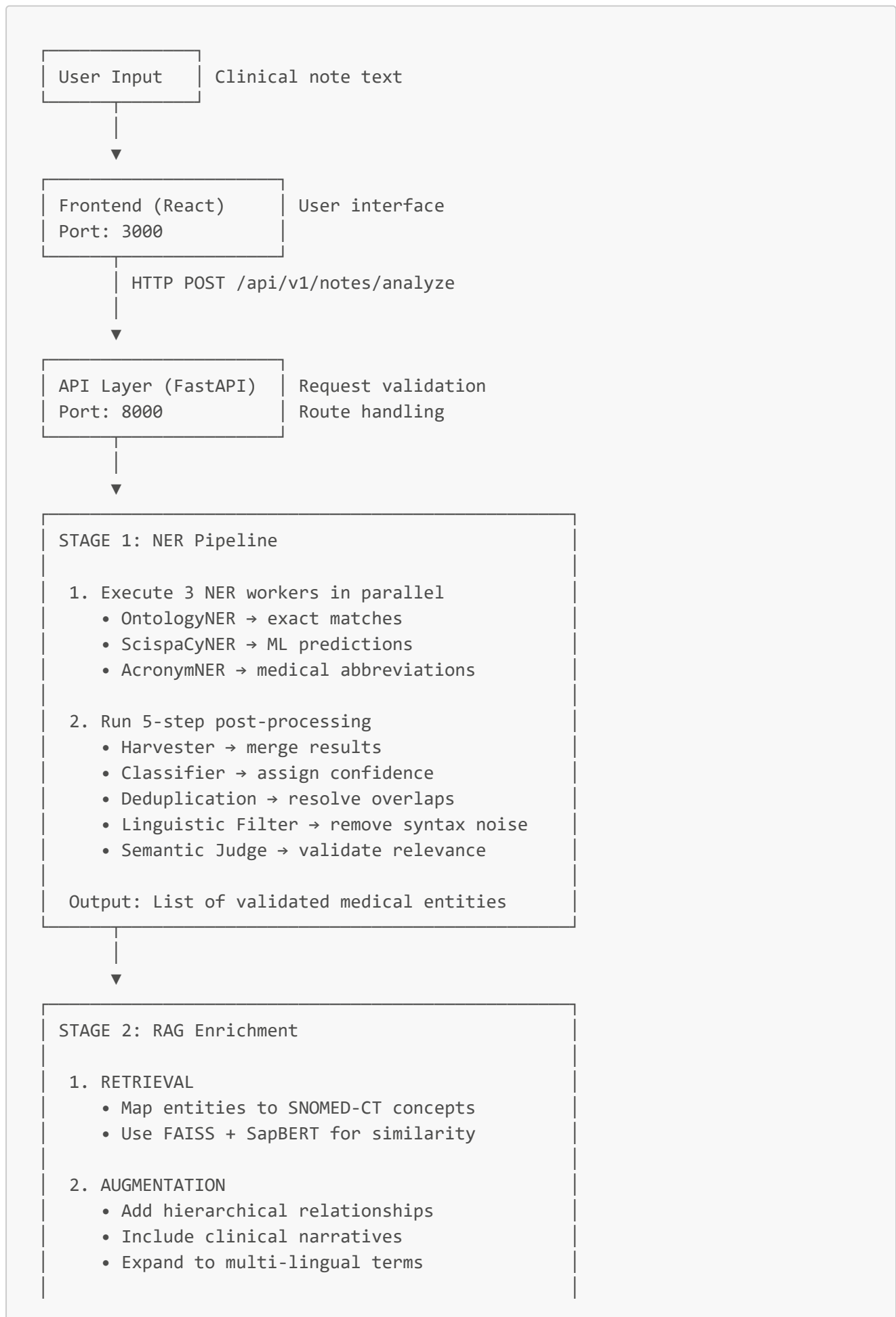
- Real-time entity extraction
- Performance metrics display
- Benchmark results visualization

4. Ontology Navigator

- Browse SNOMED-CT concepts
 - View relationships and hierarchies
 - Multi-lingual term display
-

Data Flow

End-to-End Processing



3. GENERATION

- Format as structured JSON
- Add provenance and citations

Output: Enriched entity knowledge graph



API Response

JSON with entities + context



Frontend Display

Visual entity highlighting
Interactive exploration

Configuration

Environment Variables

Create `.env` file in `ofarres/` directory:

```
# API Configuration
API_HOST=0.0.0.0
API_PORT=8000

# OpenAI (for RAG Generation phase, optional)
OPENAI_API_KEY=your_api_key_here

# Model Paths
ONTOLOGY_PATH=ontology/multilingual_ontology.json
FAISS_INDEX_PATH=assets/ontology/ontology_rag.index

# NER Configuration
NER_REGISTRY_PATH=config/ner_registry.json
```

NER Registry

File: `ofarres/backend/config/ner_registry.json`

Defines available NER workers and their configuration:

```
{
  "OntologyExact": {
    "module": "src.NER.ontology_ner",
```

```
    "class": "OntologyNER",
    "ontology_path": "ontology/multilingual_ontology.json",
    "min_length": 1
  },
  "SBert": {
    "module": "src.NER.spacy_ner",
    "class": "ScispaCyNER",
    "model_name": "en_core_sci_scibert"
  },
  "Acronyms": {
    "module": "src.NER.acronym_ner",
    "class": "AcronymNER",
    "ontology_path": "ontology/multilingual_ontology.json",
    "max_acronym_len": 8
  }
}
```

Dependencies

Backend (Python 3.12+)

Core NLP:

- spacy==3.7.5
- scispacy==0.6.2
- en_core_sci_scibert (SciBERT model)
- flashtext==2.7

Machine Learning:

- sentence-transformers==5.1.1
- torch==2.9.0
- transformers==4.49.0
- faiss-cpu==1.12.0

API Framework:

- fastapi==0.123.4
- uvicorn==0.38.0
- pydantic==2.12.3

Data Processing:

- numpy==1.26.4
- pandas==2.3.3

Frontend (Node.js 18+)

```
{
  "react": "^19.2.0",
  "react-dom": "^19.2.0",
  "axios": "^1.13.2",
  "react-router-dom": "^7.9.6",
  "lucide-react": "^0.555.0",
  "react-markdown": "^10.1.0",
  "typescript": "~5.8.2",
  "vite": "^6.2.0"
}
```

Performance Considerations

NER Stage

- **Latency:** ~500ms for typical clinical note (500 words)
- **Throughput:** Can process 100+ notes/minute
- **Bottleneck:** SciBERT inference (GPU recommended)
- **Optimization:** Batch processing for large datasets

RAG Stage

- **Latency:** ~100ms for entity linking via FAISS
- **Index Size:** ~2GB for full SNOMED-CT
- **Scalability:** Stateless design allows horizontal scaling
- **Caching:** Redis recommended for frequent concept lookups

Future Enhancements

Short-term

1. **Step 06:** Cross-Encoder Ranker for better entity disambiguation
2. **Step 07:** LLM Validator (GPT-4) for edge case validation
3. **Caching Layer:** Redis for API response caching

Medium-term

1. **Real-time Processing:** WebSocket support for streaming analysis
2. **Batch API:** Endpoint for bulk note processing
3. **Export Formats:** PDF reports, FHIR resources

Long-term

1. **Multi-language Support:** Full Spanish/Catalan analysis
2. **Clinical Decision Support:** Integration with diagnostic guidelines
3. **Federated Learning:** Privacy-preserving model updates

System Requirements

Development Environment

- **OS:** Windows 10+, macOS, Linux
- **Python:** 3.12+
- **Node.js:** 18+
- **RAM:** 16GB minimum (32GB recommended)
- **Storage:** 10GB for models and ontology data

Production Environment

- **CPU:** 8+ cores recommended
- **RAM:** 32GB+ for concurrent requests
- **GPU:** NVIDIA GPU with 8GB+ VRAM (optional, for faster inference)
- **Storage:** SSD with 20GB+ available

Monitoring and Logging

Metrics Tracked

- Request latency (p50, p95, p99)
- Entity extraction accuracy (Recall, Precision, F1)
- API error rates
- Cache hit rates
- Worker execution times

Logging

- Structured JSON logs
- Log levels: DEBUG, INFO, WARNING, ERROR
- Rotation: Daily with 30-day retention

Security Considerations

API Security

- CORS configured for specific origins
- Input validation via Pydantic
- Rate limiting (recommended for production)
- API key authentication (to be implemented)

Data Privacy

- No PHI stored on server (stateless processing)
- In-memory processing only

- Audit logging for compliance
 - HIPAA compliance considerations
-

References

Medical Ontologies

- **SNOMED-CT**: <https://www.snomed.org/>
- **UMLS**: <https://www.nlm.nih.gov/research/umls/>

Models

- **SciBERT**: <https://github.com/allenai/scibert>
- **SapBERT**: <https://github.com/cambridgeltl/sapbert>

Frameworks

- **FastAPI**: <https://fastapi.tiangolo.com/>
 - **spaCy**: <https://spacy.io/>
 - **scispaCy**: <https://allenai.github.io/scispacy/>
-

Authors

Oscar Farrés - NLP Engineer

- NER Pipeline Architecture
 - RAG Integration
 - Performance Optimization
-

License

Internal Project - All Rights Reserved